

Lesson 1 — Introduction to Node

Node.js/Express

Game Plan

- Warm-Up
- What is Node?
- Node vs Browser
- Async & the Event Loop
- CJS vs ESM
- Node Globals
- Node Docs & npm
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. Have you taken a look at the lesson/assignment material for this week, or is this your first time looking at Week 1's content?
2. On a scale of 1–5, how confident do you feel about this week's concepts?

What Is Node?

Node.js is:

- A JavaScript runtime
- Runs outside the browser
- Built on Google's V8 engine
- Not sandboxed like the browser

Think of it as:

“JavaScript with access to your whole machine.”

Why Does Node Exist?

- **Originally:** JavaScript only ran in browsers
- **Problem:** Lots of JS developers; server-side languages were complex
- **Solution:** Let JavaScript run on servers

Now used for:

- APIs
- CLIs
- Real-time apps
- Build tools
- Scripts

Node vs Browser

Browser:

- Has `window`, `document`
- Can manipulate DOM
- Cannot access file system
- Sandboxed

Node:

- No DOM
- Has `process`, `__dirname`
- Can access files
- Can open servers

Quick Think

What are 3 things you can do in Node that you cannot do in browser JS?

(Discuss for 1–2 minutes)

The Big Constraint

JavaScript is:

- Single-threaded
- One call stack
- One thing at a time

So how does Node handle:

- File reads?
- Network calls?
- Databases?

The Call Stack

- Functions go onto the stack
- Last In, First Out (LIFO)
- Runs one instruction at a time

Node cannot do two JS instructions simultaneously.

The Event Loop

When something takes time:

- It moves off the stack
- Node keeps running other code
- When ready → callback goes into queue
- Queue is FIFO
- Stack processes it when empty

Prediction Exercise

What logs first?

```
const fs = require("fs");  
  
fs.open("file.txt", "w", () => {  
  console.log("File opened");  
});  
  
console.log("Last line");
```

Answer

```
Last line  
File opened
```

Because `fs.open()` is asynchronous.

Async Patterns

Three main styles:

1. Callbacks
2. Promises
3. Async / Await

All accomplish async work.

Callback Example

```
fs.readFile("sample.txt", "utf8", (err, data) => {  
  if (err) return console.error(err);  
  console.log(data);  
});
```

Problem:

Nested callbacks → messy code.

Converting Callbacks to Promises

What if a library only supports callbacks?

Use Node's built-in **Utility** module:

```
const util = require("util");
const fs = require("fs");

// Converts callback-style to Promise-style
const readFile = util.promisify(fs.readFile);

async function run() {
  const data = await readFile("test.txt", "utf8");
  console.log(data);
}
```

Promise Example

```
fs.promises.readFile("sample.txt", "utf8")  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```


Async / Await (Preferred)

```
const fs = require("fs/promises");

async function readFile() {
  try {
    const data = await fs.readFile("sample.txt", "utf8");
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
```

We Do — Globals

What do these log?

```
console.log(__dirname);  
console.log(__filename);  
console.log(process.pid);  
console.log(process.platform);
```

What is `process` ?

Key Node Globals

- `process`
- `__dirname`
- `__filename`
- `module`
- `require()`

These do NOT exist in browser JS.

CommonJS vs ESM

Browser:

```
import { something } from "./file.js";
```

Node (CJS):

```
const something = require("./file");
```

Exports:

```
module.exports = { myFunction };
```

We will use CJS.

Node Docs & npm

Node gives you:

- Built-in modules like `fs`, `path`, `os`, `http`, `crypto`
- Official docs at <https://nodejs.org/api/>
- Third-party packages through npm

Mentor prompt:

When would you use a built-in module vs an npm package?

Streams (Big Idea)

Reading large files with `readFile()` loads everything into memory.

Streams:

- Read in chunks
- More memory efficient

Think:

Small sips, not whole bottle.

Stream Controls: highWaterMark

By default, Node buffers **64KB** for a stream.

We can control the "sip size" using `highWaterMark`.

```
const fs = require("fs");

const stream = fs.createReadStream("large_file.txt", {
  highWaterMark: 1024 // Limits chunk size to 1KB
});

stream.on("data", (chunk) => {
  console.log("Received a 1KB chunk of data");
});
```

Assignment Preview

You will:

1. Explain Node concepts in markdown
2. Log Node globals (exact formatting!)
3. Read a file using:
 - Callback
 - Promise
 - Async/Await
4. Use:
 - `os`
 - `path`
 - `fs.promises`

Important: Formatting Matters

Tests expect EXACT output.

- One space after colon
- Exact capitalization
- Exact strings

Be precise.

Promise Check

What does this return?

```
async function example() {  
  return 5;  
}
```

Promise Check Answer:

A Promise that resolves to 5.

Two Ways to Get Promise Value

1. `await`

2. `.then()`

Prefer `await`.

Wrap-Up

In chat:

1. What is the event loop?
2. What can Node do that browser JS cannot?
3. What does an async function return?

Confidence Check

On a scale of 1–5:

How comfortable do you feel starting Assignment 1?

Resources

- <https://nodejs.org/api/>
- Ask questions in Slack
- Rewatch event loop video if needed

Closing

This week:

Understand the environment.

Next week:

Start building with it.

See you then!